MOSHITHA KOLANJIYAPPAN (192421205)

CSA06 - Design and Analysis of Algorithms

Q23. Word Wrap Problem vs Job Sequencing with Deadlines (DP vs Greedy)

1. Introduction

The **Word Wrap Problem** and the **Job Sequencing with Deadlines Problem** are two important optimization problems in Design and Analysis of Algorithms (DAA). Although both require selecting an optimal sequence of decisions, they use different algorithmic strategies.

The Word Wrap Problem is commonly solved using **Dynamic Programming (DP)** because the best arrangement of words on one line depends on decisions made for subsequent lines. Its objective is to minimize the total **raggedness**, or unused space at the end of lines.

Job Sequencing with Deadlines is typically solved using a **Greedy Algorithm**. The objective is to maximize the total profit by selecting jobs that can be completed before their deadlines. A locally optimal decision—choosing a high-profit job first—can be extended to produce a globally optimal solution under the standard assumptions of the problem.

Thus, these problems provide a useful comparison between **Dynamic Programming and Greedy optimization**.

---

2. Word Wrap Problem Using Dynamic Programming

Problem Statement

Given a sequence of words with lengths:

Word lengths = [3, 2, 2, 5]

and a line width:

M = 6

we must arrange the words into lines without exceeding the maximum line width.

The objective is to minimize the total raggedness.

For a line containing words from position i to j, the number of unused spaces is:

$$
extra(i,j)=M-\left(\sum_{k=i}^{j}l_k+(j-i)\right)
$$

where:

- M = maximum line width
- l_k = length of the kth word
- (j-i) = number of spaces between the words

A common cost function is:

$$
cost(i,j)=extra(i,j)^2
$$

if the words fit on the line.

The square is used so that lines with large amounts of unused space are penalized more heavily.

---

3. Step-by-Step Word Wrap Solution

Given:

Word	Length
W1	3
W2	2
W3	2
W4	5

Line width:

$$
M=6
$$

Possible arrangements

Line 1: W1 W2

Length:

[
3+1+2=6
]

Extra space:

[
6-6=0
]

Cost:

[
0^2=0
]

Remaining words are W3 and W4.

Line 2: W3

Length:

[
2
]

Extra space:

[
6-2=4
]

Cost:

[
4^2=16
]

Line 3: W4

Length:

[
5
]

Extra space:

$$
6-5=1
$$

Cost:

$$
1^2=1
$$

Therefore:

$$
Total\ Cost=0+16+1=17
$$

However, another arrangement is:

Line 1: W1

Extra space:

$$
6-3=3
$$

Cost:

$$
3^2=9
$$

Line 2: W2 W3

Length:

$$
2+1+2=5
$$

Extra space:

[
6-5=1
]

Cost:

[
1^2=1
]

Line 3: W4

Extra space:

[
6-5=1
]

Cost:

[
1^2=1
]

Total:

[
9+1+1=11
]

Therefore, the better arrangement is:

W1
W2 W3
W4

with minimum cost:

[
\boxed{11}
]

---

4. Dynamic Programming Algorithm

Let:

[
DP[i]
]

represent the minimum cost of arranging words from position i to the end.

The recurrence is:

[
DP[i]=\min_{j\geq i}{cost(i,j)+DP[j+1]}
]

provided words i through j fit on one line.

The base condition is:

[
DP[n]=0
]

where n is the number of words.

Pseudocode

```
WORD_WRAP(words, width)

   n = number of words

   dp[n] = 0

   for i = n-1 down to 0
      dp[i] = infinity

      lineLength = 0

      for j = i to n-1

         lineLength =
            lineLength + length(words[j])

         if j > i
            lineLength = lineLength + 1

         if lineLength > width
```

```
            break

        extra = width - lineLength

        if j == n-1
            cost = 0
        else
            cost = extra * extra

        dp[i] = min(dp[i],
                    cost + dp[j+1])

    return dp[0]
```

The final line is often assigned zero cost because raggedness at the end of a paragraph is normally ignored.

---

5. Correctness of Word Wrap DP

The Dynamic Programming solution is correct because of the **optimal substructure** property.

Suppose the first line contains words i through j. Once this choice is made, the remaining words j+1 through n-1 form an independent Word Wrap problem.

If the remaining words were not arranged optimally, replacing their arrangement with a better one would reduce the total cost. This would contradict the assumption that the original solution was optimal.

Therefore:

[
Optimal(i)=\min_j
\left(LineCost(i,j)+Optimal(j+1)\right)
]

By evaluating every valid ending position j, the algorithm considers every possible first-line decision and chooses the minimum total cost.

Hence, the DP algorithm produces the optimal word arrangement.

---

6. Complexity of Word Wrap

For n words, the algorithm considers possible starting and ending positions.

Time Complexity

$$
\boxed{O(n^2)}
$$

Each pair of word positions can be examined once.

Space Complexity

$$
\boxed{O(n)}
$$

for the DP table.

If the actual line arrangement is stored using additional parent/backtracking information, the space remains generally:

$$
O(n)
$$

---

7. Job Sequencing with Deadlines Using Greedy Algorithm

Problem Statement

In Job Sequencing with Deadlines:

- Each job requires one unit of time.
- Every job has a deadline.
- Every job has a profit.
- A job earns its profit only if completed before or on its deadline.
- Only one job can be performed at a time.

The objective is:

$$
\boxed{\text{Maximize total profit}}
$$

Consider the following jobs:

Job	Deadline	Profit
J1	2	100
J2	1	19
J3	2	27
J4	1	25
J5	3	15

Maximum deadline:

[
3
]

Therefore, there are three available time slots:

Slot 1 | Slot 2 | Slot 3

---

8. Step-by-Step Greedy Solution

Step 1: Sort jobs by decreasing profit

Job	Deadline	Profit
J1	2	100
J3	2	27
J4	1	25
J2	1	19
J5	3	15

Step 2: Select J1

J1 has deadline 2.

Place it in the latest available slot before its deadline:

Slot 1 | Slot 2 | Slot 3
| J1 |

Step 3: Select J3

J3 has deadline 2.

Slot 2 is already occupied, so check slot 1.

Slot 1 | Slot 2 | Slot 3
J3 | J1 |

Step 4: Select J4

J4 has deadline 1.

Slot 1 is occupied, so J4 cannot be scheduled.

Step 5: Select J2

J2 also has deadline 1.

Slot 1 is occupied, so J2 cannot be scheduled.

Step 6: Select J5

J5 has deadline 3.

Slot 3 is available.

Slot 1 | Slot 2 | Slot 3
J3 | J1 | J5

Total profit:

[
27+100+15=142
]

Therefore:

[
\boxed{Maximum\ Profit=142}
]

---

9. Greedy Algorithm

```
JOB_SEQUENCING(jobs)

    sort jobs by decreasing profit

    maxDeadline = maximum deadline
```

```
create slots[1...maxDeadline]
mark all slots as empty

totalProfit = 0

for each job in sorted jobs

    for slot = job.deadline down to 1

        if slots[slot] is empty

            slots[slot] = job
            totalProfit =
                totalProfit + job.profit

            break

return slots, totalProfit
```

The important greedy choice is:

Select the highest-profit available job and place it in the latest possible slot before its deadline.

Placing a selected job as late as possible preserves earlier slots for jobs with tighter deadlines.

---

10. Correctness of the Greedy Algorithm

The greedy strategy is correct for the standard Job Sequencing with Deadlines problem because of the **greedy-choice property**.

Consider the job having the highest profit among the remaining jobs.

If it can be scheduled before its deadline, scheduling it does not reduce the ability to schedule other jobs when it is placed in the **latest available slot**.

If another lower-profit job occupies that slot, replacing that lower-profit job with the higher-profit job cannot decrease total profit.

Furthermore, placing the selected job as late as possible leaves earlier slots available for jobs with smaller deadlines.

By repeatedly making this choice, the algorithm constructs an optimal schedule.

Thus, under the standard assumptions—unit execution times, independent jobs, known deadlines, and fixed profits—the greedy algorithm gives the maximum possible profit.

---

11. Complexity of Job Sequencing

Let:

- n = number of jobs
- D = maximum deadline

Sorting the jobs requires:

$$
O(n\log n)
$$

Searching for an available slot may require up to D operations for every job:

$$
O(nD)
$$

Therefore:

$$
\boxed{O(n\log n+nD)}
$$

Since normally:

$$
D\leq n
$$

the worst-case complexity is:

$$
\boxed{O(n^2)}
$$

Space Complexity

The slot array requires:

$$
\boxed{O(D)}
$$

Additional space for sorting depends on the implementation.

---

12. Comparison: Word Wrap vs Job Sequencing

Feature	Word Wrap	Job Sequencing
Technique	Dynamic Programming	Greedy
Main objective	Minimize raggedness	Maximize profit
Decision	Where to break lines	Which job to schedule
Input	Word lengths and line width	Jobs, deadlines and profits
Optimization type	Minimization	Maximization
Main principle	Optimal substructure	Greedy-choice property
Recurrence	DP[i] = min(cost + DP[j+1])	No DP recurrence
Sorting required	No	Yes, by profit
Typical time	$O(n^2)$	O(n log n + nD)
Space	O(n)	O(D)
Output	Optimal line arrangement	Optimal job schedule
Real-world use	Text formatting	Scheduling resources

---

13. Difference in Objective Functions

The most important difference is their optimization objective.

Word Wrap

Word Wrap minimizes visual irregularity:

$$
\boxed{\min \sum extra_i^2}
$$

A line with a large amount of unused space receives a high penalty.

For example:

This is a
sample

paragraph

The goal is to create visually balanced lines.

Job Sequencing

Job Sequencing maximizes financial or operational benefit:

$$
\boxed{\max \sum Profit_i}
$$

For example, if jobs have profits of 100, 27 and 15, selecting the correct combination gives the highest possible total profit.

Therefore:

Word Wrap → minimize cost

Job Sequencing → maximize benefit

---

14. Why DP Is Used for Word Wrap

A simple greedy strategy such as "put as many words as possible on each line" does not always produce the minimum total raggedness.

For example, filling the current line as much as possible may leave an extremely short final line. The locally best-looking line can therefore produce a poor overall arrangement.

Dynamic Programming avoids this problem by considering multiple possible line endings and storing the best solution for each suffix of the word sequence.

This eliminates repeated calculations and guarantees an optimal result.

---

15. Why Greedy Works for Job Sequencing

Job Sequencing has a special structure that allows greedy optimization.

Choosing a high-profit job first is beneficial, provided it can be scheduled before its deadline. Placing it as late as possible protects earlier time slots.

The algorithm does not need to explore every possible combination of jobs because the greedy-choice property guarantees that the selected high-profit job can be part of an optimal schedule.

This is a major advantage of greedy algorithms: when their correctness conditions hold, they can solve an optimization problem more simply than DP.

---

16. Real-World Applications

Word Wrap Applications

Word Wrap algorithms are used in:

- Word processors
- Text editors
- Web browsers
- Mobile applications
- PDF/document generation
- Report formatting
- User-interface text rendering
- Publishing and document layout systems

For example, a document editor must determine where lines should end so that text appears visually balanced.

Job Sequencing Applications

Job Sequencing is useful in:

- Manufacturing
- CPU and resource scheduling
- Production planning
- Project management
- Advertising campaigns
- Cloud computing
- Telecommunications
- Delivery planning

For example, a company may have several tasks with deadlines and different expected profits. Scheduling the highest-value feasible tasks can maximize revenue.

---

17. Optimization Strategies

Several strategies can improve practical implementations.

Word Wrap

1. **Prefix sums** can calculate line lengths efficiently.
2. **Dynamic Programming** avoids repeated subproblem calculations.
3. **Backtracking/parent arrays** can reconstruct the actual line breaks.
4. The cost function can be changed depending on application requirements.
5. Memory usage can be optimized if only the minimum cost is required.

Job Sequencing

1. Sort jobs by decreasing profit.
2. Always place jobs in the latest available slot.
3. Use a Disjoint Set Union (DSU) structure to find available slots more efficiently.
4. Avoid scanning every slot when deadlines are large.
5. Handle deadline constraints before scheduling.

With an efficient DSU-based implementation, slot assignment can be significantly faster than scanning backward through every slot.

---

18. Key Insights

The comparison demonstrates an important principle in algorithm design:

Not every optimization problem should be solved using the same strategy.

Word Wrap has overlapping subproblems and optimal substructure, but a simple local choice is not sufficient to guarantee the globally minimum raggedness. Therefore, Dynamic Programming is appropriate.

Job Sequencing has a strong greedy-choice property under its standard assumptions. Therefore, sorting jobs by profit and scheduling each in the latest available slot produces an optimal solution.

The choice between DP and Greedy should therefore depend on the mathematical structure of the problem rather than simply on which algorithm appears faster.

---

19. Conclusion

Word Wrap and Job Sequencing illustrate two different approaches to optimization.

The **Word Wrap Problem** uses Dynamic Programming to minimize the total raggedness of a paragraph. Its recurrence considers all possible positions at which a line can end and combines the current line cost with the optimal cost of the remaining words. Its typical time complexity is $\mathbf{O(n^2)}$ and space complexity is $\mathbf{O(n)}$.

The **Job Sequencing with Deadlines Problem** uses a Greedy approach to maximize total profit. Jobs are sorted in decreasing order of profit and placed in the latest available slot before their deadlines. Its typical complexity is $\mathbf{O(n \log n + nD)}$, becoming $\mathbf{O(n^2)}$ in the worst case when $D \leq n$, with $\mathbf{O(D)}$ scheduling space.

The fundamental distinction is therefore:

$$
\boxed{\text{Word Wrap: DP + Minimize Cost}}
$$

$$
\boxed{\text{Job Sequencing: Greedy + Maximize Profit}}
$$

Both algorithms demonstrate that understanding **objective functions, optimal substructure, greedy-choice properties, constraints, and computational complexity** is essential for selecting the right optimization technique. In real-world systems, the most effective solution often combines the theoretical algorithm with practical improvements such as efficient data structures, preprocessing, and application-specific cost functions.